

Day 47



Hoisting in JavaScript:

Hoisting means:

JavaScript moves variable declarations and function declarations to the top of their scope (before code runs). This allows you to use a variable or function before writing it in the code — but with different behaviors depending on var, let, const, and function types.

How Hoisting Works:

JavaScript has two phases:

1. Creation Phase (Memory Allocation Phase)

- JS scans the entire code.
- It allocates memory for:
 - var declarations → set to undefined
 - let/const → memory reserved but not initialized (they stay in the “Temporal Dead Zone”)
 - function declarations → stored as complete functions

2. Execution Phase

- Code runs line by line.
- Variables get their assigned values during execution.

Why Hoisting Exists?

It helps JavaScript:

- scan code first,
- allocate memory,
- make functions available early.

It is part of JavaScript’s Execution Context creation.

Example:

```
JS main.js > greet
1  greet();
2
3  function greet() {
4    |   console.log("Hello, World!");
5  }

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORT

PS E:\JavaScript\Day41-50\Day47> node main.js
Hello, World!
```

Example: variable is declared later but used earlier, so gives 'undefined' as output.

```
7 console.log(a);
8 greet();
9 function greet() {
10     console.log("Hello, World!");
11 }
12 var a = 10;
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS E:\JavaScript\Day41-50\Day47> node main.js
undefined
Hello, World!

Example: let is used in above example.

```
7 console.log(a);
8 greet();
9 function greet() {
10     console.log("Hello, World!");
11 }
12 let a = 10;
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS E:\JavaScript\Day41-50\Day47> node main.js
E:\JavaScript\Day41-50\Day47\main.js:7
console.log(a);
^
ReferenceError: Cannot access 'a' before initialization

Closures in JavaScript:

What Is a Closure in JavaScript?

A closure is created when a function remembers and can access variables from its outer (parent) function—even after that outer function has finished executing.

Think of a closure as: A backpack that a function carries with it, containing the variables from where it was created.

In simple words:

- A function inside another function
- Inner function keeps access to the outer function's variables
- Even when the outer function is no longer active

Why Closures Happen:

Closures happen because of how JavaScript works:

- JS creates a scope chain
- Inner functions keep a reference to their outer scope
- Nothing is deleted until JS sees no references

How Closures Work:

Step 1: You create a function inside another function

Step 2: The inner function uses a variable from the outer function

Step 3: You return the inner function

Step 4: Even after the outer function finishes, the returned inner function still remembers the outer variables.

Why Closures Are Useful

- Data privacy (private variables)
- Functions that remember state
- Memoization
- Event handlers
- Callback functions
- Module pattern

Example:

```
JS closures.js > ...
1  function outer() {
2      let count = 0;
3
4      function inner() {
5          count++;
6          console.log(count);
7      }
8
9      return inner;
10 }
11
12 const fn = outer();
13 fn(); // 1
14 fn(); // 2
15 fn(); // 3
16
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS E:\JavaScript\Day41-50\Day47> node .\closures.js
PS E:\JavaScript\Day41-50\Day47> node .\closures.js
1
2
3
```

Why does this work?

Even though `outer()` finished, `inner()` still remembers `count` → closure.

--The End--